Output – Questions 18 to 32

Question 18. Increase commission of all salespersons by 200

```
UPDATE Salespeople SET commission = commission + 200;
SELECT * FROM Salespeople;
```

Output

snum	sname	city	commission
1001	Gaurav	Pune	212
1002	Gopal	Mumbai	210
1003	Geeta	Delhi	215
1004	Ramesh	Pune	211
1005	Ganesh	Nagar	213
1006	Gyan	Mumbai	209
1007	Amit	Delhi	214
1008	Rajesh	Nagar	210

Question 19. Display each order number followed by the name of customer who made it

```
SELECT o.onum, c.cname
FROM Orders o JOIN Customers c ON o.snum = c.snum
ORDER BY o.onum;
```

Output

onum	cname
3001	Pooja
3001	Rahul
3002	Gopal
3002	Priya
3003	Kiran
3003	Vikas
3004	Pooja
3004	Rahul
3005	Gopal
3005	Priya
3006	Rohit
3006	Suresh
3007	Neha
3008	Amit
3008	Meena
3009	Rohit
3009	Suresh
3010	Pooja
3010	Rahul
3011	Gopal
3011	Priya
3012	Kiran
3012	Vikas
3013	Amit
3013	Meena
3014	Neha
3015	Pooja
3015	Rahul

Question 20. Salesperson commission on each order by a customer with rating above 100

```
SELECT o.onum, c.cname, c.rating, o.amt * s.commission / 100 AS commission_amt
FROM Orders o JOIN Customers c ON o.snum = c.snum
JOIN Salespeople s ON o.snum = s.snum
WHERE c.rating > 100;
```

Output

onum	cname	rating	commission_amt
3001	Pooja	180	600
3001	Rahul	150	600
3002	Gopal	250	750
3003	Kiran	200	675
3004	Pooja	180	360
3004	Rahul	150	360
3005	Gopal	250	250
3006	Rohit	120	780
3007	Neha	300	810
3008	Meena	220	880
3009	Rohit	120	455
3010	Pooja	180	504
3010	Rahul	150	504
3011	Gopal	250	550
3012	Kiran	200	420
3013	Meena	220	352
3014	Neha	300	585
3015	Pooja	180	840
3015	Rahul	150	840

Question 21. Sub-query: all orders for the customer named 'Gopal'

```
SELECT * FROM Orders
WHERE snum IN (SELECT snum FROM Customers WHERE cname = 'Gopal');
```

Output

onum	odate	snum	amt
3002	2005-02-05	1002	7500
3005	2005-02-06	1002	2500
3011	2005-02-09	1002	5500

Question 22. Names & ratings of customers who have above-average orders

```
SELECT cname, rating FROM Customers
WHERE snum IN (SELECT snum FROM Orders
  WHERE amt > (SELECT AVG(amt) FROM Orders));
```

Output

cname	rating
Gopal	250
Rahul	150
Priya	90
Amit	80
Suresh	70
Neha	300
Rohit	120
Pooja	180
Meena	220

Question 23. Union: names, cities & ratings with 'high rating' / 'low rating'

```
SELECT cname, city, rating, 'high rating' AS category FROM Customers WHERE rating >= 200
UNION
SELECT cname, city, rating, 'low rating' FROM Customers WHERE rating < 200;
```

Output

cname	city	rating	category
Amit	Pune	80	low rating
Arjun	NULL	95	low rating
Gopal	Mumbai	250	high rating
Kiran	Delhi	200	high rating
Meena	Pune	220	high rating
Neha	Mumbai	300	high rating
Pooja	NULL	180	low rating
Priya	Mumbai	90	low rating
Rahul	Pune	150	low rating
Rohit	Nagar	120	low rating
Suresh	Nagar	70	low rating
Vikas	Delhi	60	low rating

Question 24. Name & number of each salesperson & customer with more than one current order

```
SELECT s.snum AS number, s.sname AS name FROM Salespeople s
WHERE s.snum IN (SELECT snum FROM Orders GROUP BY snum HAVING COUNT(*) > 1)
UNION
SELECT c.cnum, c.cname FROM Customers c
WHERE c.snum IN (SELECT snum FROM Orders GROUP BY snum HAVING COUNT(*) > 1)
ORDER BY name;
```

Output

number	name
2005	Amit
1005	Ganesh
1001	Gaurav
1003	Geeta
1002	Gopal
2001	Gopal
1006	Gyan
2004	Kiran
2011	Meena
2007	Neha
2009	Pooja
2003	Priya
2002	Rahul
1004	Ramesh
2008	Rohit
2006	Suresh
2010	Vikas

Question 25. Index so each salesperson can retrieve orders grouped by date quickly

```
CREATE INDEX idx_orders_snum_date ON Orders(snum, odate);
```

Output

Result
Index idx_orders_snum_date created successfully.

Question 26. View of customers who have the highest rating

```
CREATE VIEW top_rated_customers AS
SELECT * FROM Customers
WHERE rating = (SELECT MAX(rating) FROM Customers);

SELECT * FROM top_rated_customers;
```

Output

cnum	cname	city	rating	snum
2007	Neha	Mumbai	300	1006

Question 27. View showing number of salespeople in each city

```
CREATE VIEW salespeople_per_city AS
SELECT city, COUNT(*) AS num_salespeople
FROM Salespeople GROUP BY city;

SELECT * FROM salespeople_per_city;
```

Output

city	num_salespeople
Delhi	2
Mumbai	2
Nagar	2
Pune	2

Question 28. PL/SQL: display a number in reverse order

```
DECLARE
  n NUMBER := 12345; r NUMBER := 0; d NUMBER;
BEGIN
  WHILE n > 0 LOOP
    d := MOD(n, 10);
    r := r * 10 + d;
    n := TRUNC(n / 10);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Reverse = ' || r);
END;
```

Output

Output

```
Reverse = 54321

PL/SQL procedure successfully completed.
```

Question 29. PL/SQL: factorial of a given number

```
DECLARE
  n NUMBER := 5; f NUMBER := 1;
BEGIN
  FOR i IN 1..n LOOP
    f := f * i;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Factorial of ' || n || ' = ' || f);
END;
```

Output

Output

```
Factorial of 5 = 120

PL/SQL procedure successfully completed.
```

Question 30. PL/SQL: generate Fibonacci series (first 10 terms)

```
DECLARE
  a NUMBER := 0; b NUMBER := 1; c NUMBER; s VARCHAR2(200) := '0 1';
BEGIN
  FOR i IN 3..10 LOOP
    c := a + b;
    s := s || ' ' || c;
    a := b; b := c;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Fibonacci series: ' || s);
END;
```

Output

Output

Fibonacci series: 0 1 1 2 3 5 8 13 21 34

PL/SQL procedure successfully completed.

Question text reads "3utest3at series" (garbled); interpreted as Fibonacci series.

Question 31. PL/SQL: area of circle for radius 3 to 7, stored in table areas

```
CREATE TABLE areas (radius NUMBER(2), area NUMBER(8,2));

DECLARE
  pi CONSTANT NUMBER := 3.14159265;
  r NUMBER := 3;
BEGIN
  WHILE r <= 7 LOOP
    INSERT INTO areas VALUES (r, ROUND(pi * r * r, 2));
    r := r + 1;
  END LOOP;
  COMMIT;
END;

SELECT * FROM areas;
```

Output

radius	area
3	28.27
4	50.27
5	78.54
6	113.1
7	153.94

Question 32. PL/SQL: deduct Rs.100 from balance if below minimum balance (table acct)

```
-- acct(acct_no, balance); minimum balance = 500
DECLARE
  v_acct  acct.acct_no%TYPE := &acct_no;   -- user enters 101
  v_bal   acct.balance%TYPE;
  min_bal CONSTANT NUMBER := 500;
BEGIN
  SELECT balance INTO v_bal FROM acct WHERE acct_no = v_acct;
  IF v_bal < min_bal THEN
    UPDATE acct SET balance = balance - 100 WHERE acct_no = v_acct;
    DBMS_OUTPUT.PUT_LINE('Balance below minimum. Rs.100 deducted.');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Balance is sufficient. No deduction.');
  END IF;
  COMMIT;
END;

SELECT * FROM acct;
```

Output

acct_no	balance
101	250
102	1200
103	800

Sample data before: (101, 350), (102, 1200), (103, 800). Entered account 101 → balance 350 < 500, so Rs.100 deducted.